Verifiable Two-Way Private Information Retrieval

***Abstract*—XYZ**

***Index Terms*—AAA,BBB,CCC**

I. Problem Statement

$data_id_i$ refers to the transaction ID, $group_id_i$ refers to the user ID, val_i refers to the payment amount, and t_i refers to the timestamp of such transaction request. Let $\mathcal{S}$ be a server holding n vectors $\mathbf{v}_i = (data_id_i, group_id_i, val_i, t_i)$ for $i = 1, \cdots, n$. Here val_i is the actual data to be analyzed, where it is assigned to a specific data ID and group ID where n vectors have pairwise distinct data IDs. t_i represents the time when the data is generated. For a data point $\mathbf{v}$, we denote by $\mathbf{v}.id, \mathbf{v}.gid, \mathbf{v}.val$, and $\mathbf{v}.t$, its data ID, group ID, value, and timestamp, respectively. We denote by $S = \{\mathbf{v}_1, \cdots, \mathbf{v}_n\}$ and for any group ID gid^*, we denote by $S^{(gid^*)} = \{\mathbf{v} \in S : \mathbf{v}.gid = gid^*\}$. For a data ID did, we denote by $did.gid$, the group ID assigned to the data ID did. Lastly, for a timestamp T, denote by S_T and $S_T^{(gid^*)}$, the subsets of S and $S^{(gid^*)}$ containing the data with timestamps more recent than T.

Here we can see that such n data refers to the existing transactions that have been previously approved.

When a new transaction comes in, a wallet owner contacts the server, which is done by the wallet operator to generate its transaction ID, which is linked by the wallet operator to the wallet owner's ID. Here the relation can be established either by having the transaction IDs be valid commitments of the wallet owner's ID or by the wallet operator holding a table providing the link between the user ID and the transaction ID. Once the wallet owner receives the transaction ID, which is denoted as $\widehat{tid}$, the payment request can be done.

Once the payment request is done, the client, which is done by Alipay+, hold $\mathbf{x} \triangleq (\widehat{tid}, v, T, f)$ where $v \in \mathcal{V}$ is the amount to be sent. Here, before payment can be approved, some verification needs to be done. More specifically, let $S^{(T,0)}$ contains all the transactions done by the requester with timestamp more recent that T and $S^{(T)} = S^{(T,0)} \cup \{v\}$ where it is treated as a multiset. Then for the payment to be approved, $S^{(T)}$ needs to satisfy the requirement that $f(S^{(T)}) = 0$. In one use case we are considering, T is set to be 1 year and we require that the total amount of payment that has been done (including the requested amount) is within a publicly determined threshold τ.

The overall functionality can be found in Figure 1.

Given such inputs, the client requires the following:

1) Proof of the correct generation of $S_T^{(\widehat{did}.gid)}$ from S.
2) Proof of the correct calculation of $\mathbf{y} \triangleq f\left(S_T^{(\widehat{did}.gid)} \cup \{v\}\right)$

On the other hand, the server requires

1) The client not to obtain any information regarding the group ID based on $\widehat{did}$ or other information except from what can be inferred from $\mathbf{y}$. In particular, the client should have negligible probability of correctly concluding whether $\mathbf{v}_i \in S_T^{(\widehat{gid})}$ for $i = 1, \cdots, n$.
2) Client does not know $\widehat{gid}$. Instead, it holds some random string that the server can use to identify the desired $\widehat{gid}$.

II. Approach

Assume the user is the wallet owner, the client is Alipay+, and the server is PayPay. We assume the following components:

1) **Identity binding commitment**: Let $\mathcal{I}, \mathcal{V}, \mathcal{R}$, and $\mathcal{C}$ be the spaces of identity, value, randomness, and commitment respectively.
   Given an ID $id \in \mathcal{I}$ and a value $v \in \mathcal{V}$, the (ID,value) commit $\Pi.\texttt{Commit}$ uses a randomness $r \in \mathcal{R}$ to generate a random commitment $c_{id,v}$ of the pair such that it cannot open to a different pair (id', v') except with negligible probabilty. The commitment scheme should be hiding and binding.
   Given such commitment scheme, we require the following functionalities.
   a) *ID verification*: Consider two (ID,value) commitments c_1 and c_2 for pairs (id_1, val_1) and (id_2, val_2) respectively. We require the following zero-knowledge proof protocol $\Pi.\texttt{IDEq}$. If $id_1 = id_2$, then the prover can convince a verifier using $\Pi.\texttt{IDEq}$ of such fact. If $id_1 \triangleq id_2$, then the verifier will reject except with a negligible probability.
   b) *Value verification*: Suppose that the prover and the verifier learns a common value $v \in \mathcal{V}$ while the prover further learns an ID $id \in \mathcal{I}$. Suppose the prover generates an (ID,value) commitment $c_1 = c_{id,v}$. We require the following zero-knowledge proof protocol $\Pi.VVer$ for the prover to convince the verifier that the value committed to c_1 is indeed v.
2) **Secure Verifiable Shuffle** Given verified commitments $c_1, \cdots, c_n$, of values $(x_1, id_1), \cdots, (x_n, id_n)$ and a permutation $\Pi : [n] \to [n]$ chosen by the server and secret from the client, generate a new set $c'_1, \cdots, c'_n$ that is verified to satisfy the following requirement: for any $i \in [n]$, c_i is the commitment of the same pair as $c'_{\Pi(i)}$.
3) **Oblivious Verified update**: Given $c_1, \cdots, c_n$, commitments of $(x_1, id_1), \cdots, (x_n, id_n)$ and another commitment d of (y, id_i), generate new $(c'_1, \cdots, c'_n)$ and convince the verifier that for $j \triangleq i$, c'_j is a commitment of (x_j, id_j) and c'_i is a commitment of $(x_i + y, id_i)$ without revealing any other values. May be achieved by ZK-SNARK
4) **Verified To-Do List:** A component that keeps a verified log of authorized actions to be done at a later date to perform an oblivious verified update to allow for a sliding window.

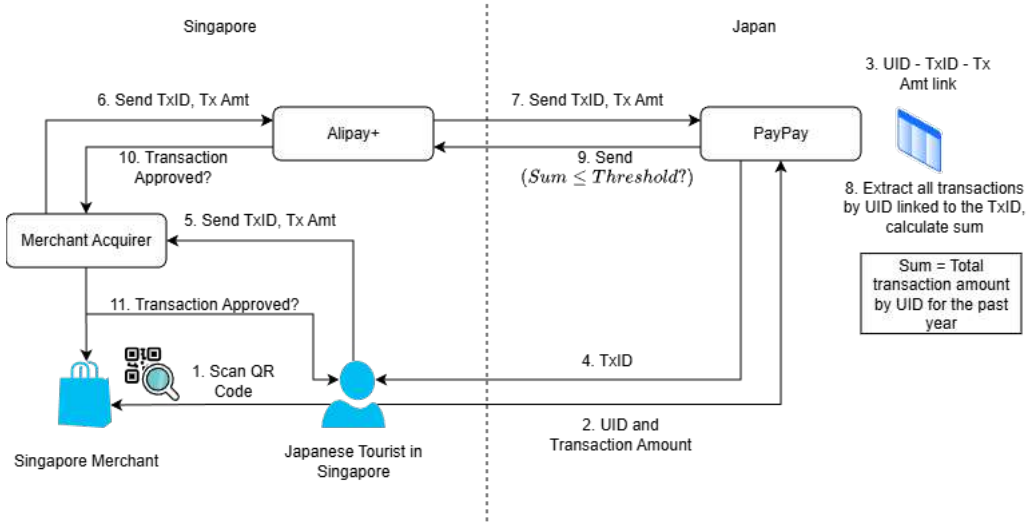


Fig. 1: Payment Verification Scenario

A. Proposed Flow

Next, we consider the general flow of our proposed approach. First, we consider the case when $\widehat{tid} = \Pi^{(\mathtt{iden})}.\mathtt{Commit}(uid)$ where uid is the user ID of the wallet owner. Recall that the client holds $\mathbf{x} = (\widehat{tid}, v, T, f)$ and the server holds several lists. Firstly, it holds the list of (uid, tid). Note that here if tid is a commitment of uid, then such list contains the randomness used for such commitment. Secondly, it also holds the list containing all approved transactions from the previous requests $(uid, txid, v, T)$. Note that this list may not be an essential component to our protocol but it provides the server with the complete log of all approved transaction that they will want to keep. Thirdly,

1)

III. Identity Binding Commitment

First we formalize the definition of identity binding commitment (IBC for short). Then we propose a scheme satisfying our requirement.

A. Definition of Identity Binding Commitment

Definition 3.1 (Identity-binding commitment scheme): Let λ be the security parameter and let $\mathcal{I}, \mathcal{V}, \mathcal{R}$, and C denote the spaces of identities, values, randomness, and commitments, respectively. An *identity-binding commitment scheme* (or IBC for short) is a tuple of PPT algorithms

$$\Pi = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{Verify})$$

defined as follows.

- **Setup:** $\mathsf{pp} \leftarrow \Pi.\mathsf{Setup}(1^\lambda)$ outputs public parameters pp.
- **(ID,value) commitment:** $c \leftarrow \Pi.\mathsf{Commit}(\mathsf{pp}|id, v; r)$ where $id \in \mathcal{I}$, $v \in \mathcal{V}$, and $r \in \mathcal{R}$.
- **Verification:** $b \leftarrow \Pi.\mathsf{Verify}(\mathsf{pp}|c, id, v; r)$ for $c \in C, id \in \mathcal{I}, v \in \mathcal{V}, r \in \mathcal{R}$. The algorithm outputs $b \in \{0, 1\}$.

Next, we define the properties that an IBC scheme has.

1) **Correctness:** For all security parameter λ, the generated public parameter $\mathsf{pp}, id \in \mathcal{I}, v \in \mathcal{V}, r \in \mathcal{R}$, if $c = \Pi.\mathsf{Commit}(\mathsf{pp}|id, v; r)$, then

$$\Pr[\mathsf{Verify}(\mathsf{pp}|c, x; r) = 1] = 1.$$

2) **Binding:** Let λ be a security parameter, $\mathsf{pp} \leftarrow \Pi.\mathsf{Setup}(1^\lambda), id \in \mathcal{I}, v \in \mathcal{V}$, and $r \in \mathcal{R}$. Let $c = \Pi.\mathtt{Commit}(\mathsf{pp}|id, v; r)$. Consider a PPT adversary $\mathcal{A}$ which is given the values (id, v, r, c) and outputs (id', v', r') such that $(id, v, r) \triangleq (id', v', r')$. Then

$$\Pr[\Pi.\mathsf{Verify}(\mathsf{pp}|c, id', v'; r') = 1] \leq \mathsf{negl}(\lambda).$$

3) **Hiding:**For $b = 0, 1$, let $id_b \in \mathcal{I}$ and $v_b \in \mathcal{V}$. We also let $r_b \in \mathcal{R}$. Lastly, define $c_b = \Pi.\mathsf{Commit}(\mathsf{pp}|id_b, v_b; r_b)$. Then for any PPT distinguisher $\mathcal{D}$ that is given c_i for a random $i = 0, 1$, the probability that $\mathcal{D}$ correctly guesses the value of i is negligible. That is,

$$|\Pr[\mathcal{D}(c_0) = 1] - \Pr[\mathcal{D}(c_1) = 1]| \leq \mathsf{negl}(\lambda).$$

We further require two functionalities for Π.

Definition 3.2 (ID-equality proof functionality): For $b = 0, 1$, let c_b be an (id, value) commitment of (id_b, v_b) with randomness r_b such that $(id_0, v_0, r_0) \triangleq (id_1, v_1, r_1)$.

For any of such pair (c_0, c_1), we say that it belongs to the relation $\mathcal{R}_{\mathsf{IDEq}} \subseteq C \times C$ if $id_0 = id_1$.

We require our IBC scheme Π to have an *ID-equality proof system* $\Pi.\mathsf{IDEq} = (\Pi.\mathsf{IDEqProve}, \Pi.\mathsf{IDEqVerify})$ to prove the relation $\mathcal{R}_{\mathsf{IDEq}}$ such that the proof system is complete, sound, and zero-knowledge.

Definition 3.3 (Value-consistency proof functionality): Let $v \in \mathcal{V}$ be known by both a prover and a verifier. Let c be the (ID,value)-commitment of (id, v) for some $id \in \mathcal{I}$. We require the proof system $\Pi.\mathsf{VVer} = (\Pi.\mathsf{VVerProve}, \Pi.\mathsf{VVerVerify})$ that can be used by the prover and verifier to convince the verifier that c is indeed a valid commitment for the value v. Such system needs to be complete, sound, and zero-knowledge.

B. Proposed Identity-Binding Commitment

In this section, we propose an IBC scheme following the definition and requirements provided in Section III-A. Our proposed scheme is based on the Pedersen commitment.

Let $\mathbb{G}$ be a group of order p for a sufficiently large prime p. We further define three pairwise distinct generators $g_{\mathsf{type}} \in \mathbb{G}$ for $\mathsf{type} \in \{\mathsf{iden}, \mathsf{val}, \mathsf{ran}\}$. Here we assume that $\mathcal{I}, \mathcal{V}, \mathcal{R}$ can be embedded to $\mathbb{Z}_p$, and $\mathcal{C} = \mathbb{G}$. Given $id \in \mathcal{I}$ and $v \in \mathcal{V}$, to commit to (id, v), we uniformly sample $r \in \mathcal{R}$ and define $c^{(\mathsf{pair})}_{(id,v)} = g^{id}_{\mathsf{iden}} g^{v}_{\mathsf{val}} g^{r}_{\mathsf{ran}}$. Here without any knowledge of the relation between the three generators, it is easy to see that the (ID,value) commitment is hiding and binding.

Now we consider the ID-equality proof functionality. As described in Definition 3.2, suppose that $(id_0, v_0, r_0) \triangleq (id_1, v_1, r_1) \in \mathcal{I} \times \mathcal{V} \times \mathcal{R}$ and let $c_b = \Pi.\mathsf{Commit}(\mathsf{pp}|id_b, v_b; r_b)$. We define $\Pi.\mathsf{IdEq}$ in a similar way as standard Pedersen commitment based proof protocol, which can be found in Algorithm 1.

To add: completeness, soundness, and ZK proofs

Next we consider the value consistency proof functionality. Recall that P and V holds $v \in \mathcal{V}$. Furthermore, P holds an ID $id \in \mathcal{I}$. Given id and v, P calculates a commitment c for (id, v) with randomness $r \in \mathcal{R}$. The objective is for P to convince V that the value being committed in c is indeed v. We again have $\Pi.\mathsf{VVer}$ to be defined in a similar was as standard Pedersen commitment-based proof protocol, which can be found in Algorithm 2.

To add: completeness, soundness, and ZK proofs

References

Protocol 1 $\Pi.\mathsf{IdEq}$

Parameters: Security parameter λ
Inputs: (ID,value) commitments c_b with randomness r_b for ID id_b and value v_b for $b = 0, 1$
Output: If $id_0 = id_1$, the verifier accepts with probability 1.

Prover		**Verifier**
1: Randomly sample $\alpha_{\mathsf{val}}, \alpha_{\mathsf{ran}} \in \mathbb{Z}_p$.	$t = g_{\mathsf{val}}^{\alpha_{\mathsf{val}}} g_{\mathsf{ran}}^{\alpha_{\mathsf{ran}}}$	
	$\longrightarrow$	
	β	2: Randomly sample $\beta \in \mathbb{Z}_p$
	$\longleftarrow$	
	$z_{\mathsf{val}} = \beta(v_0 - v_1) + \alpha_{\mathsf{val}}, z_{\mathsf{ran}} = \beta(r_0 - r_1) + \alpha_{\mathsf{ran}}$	
	$\longrightarrow$	
		3: **if** $g_{\mathsf{val}}^{z_{\mathsf{val}}} g_{\mathsf{ran}}^{z_{\mathsf{ran}}} == \mathfrak{t}(c_0/c_1)^{\beta}$ **then**
		4: Return 1 ▷ the verification is successful, $id_0 = id_1$.
		5: **else**
		6: Return 0 ▷ the verification failed

Protocol 2 $\Pi.\mathsf{VVer}$

Parameters: Security parameter λ
Inputs: The prover holds $id, v, r, c = g_{\mathsf{iden}}^{id} g_{\mathsf{val}}^{v} g_{\mathsf{ran}}^{r}$, the verifier holds v', c
Output: If $v = v'$, the verifier accepts with probability 1.

Prover		**Verifier**
1: Randomly sample $\alpha_{\mathsf{iden}}, \alpha_{\mathsf{ran}} \in \mathbb{Z}_p$.	$t = g_{\mathsf{iden}}^{\alpha_{\mathsf{iden}}} g_{\mathsf{ran}}^{\alpha_{\mathsf{ran}}}$	
	$\longrightarrow$	
	β	2: Randomly sample $\beta \in \mathbb{Z}_p$
	$\longleftarrow$	
	$z_{\mathsf{iden}} = \beta \cdot id + \alpha_{\mathsf{iden}}, z_{\mathsf{ran}} = \beta \cdot r + \alpha_{\mathsf{ran}}$	
	$\longrightarrow$	
		3: **if** $g_{\mathsf{iden}}^{z_{\mathsf{iden}}} g_{\mathsf{ran}}^{z_{\mathsf{ran}}} == \mathfrak{t}(c \cdot g_{\mathsf{val}}^{-v})^{\beta}$ **then**
		4: Return 1 ▷ the verification is successful, $id_0 = id_1$.
		5: **else**
		6: Return 0 ▷ the verification failed